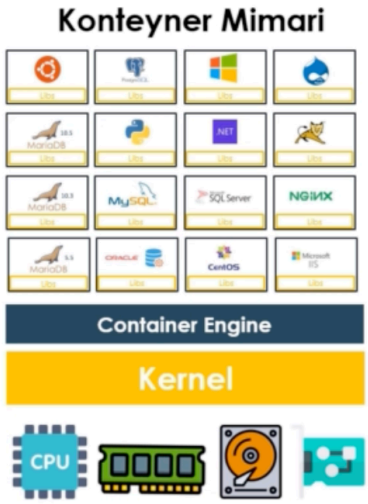






13. Gün staj

1. Bölüm: Docker Neden Ortaya Çıktı?

1.1 Geleneksel Mimari

1.2 Sanallaştırma (Hypervisor ve VM)

1.3 Sanallaştırmanın Sorunları

1.4 Container Mimarisine Geçiş

1.5 Sonuç Mesajı

2. Bölüm: Container (Docker) Nedir?

2.1 Container Kavramı

2.2 Container Neden Kullanılır

2.3 VM ile Container Farkı

2.4 Container'ın Avantajları

2.5 Docker vs VMware

3. Bölüm: Container Engine ve Docker

3.1 Container Engine

3.2 Docker Nedir

3.3 Docker Mimarisi

3.4 Namespaces ve cgroups

3.5 Docker Komutları

1. Geleneksel Mimari (Traditional Architecture)

Eskiden bir fiziksel sunucu üzerinde sadece **tek bir işletim sistemi** çalışırdı. Akış şöyleydi:

Application (Uygulama)
↓
Operating System (İşletim Sistemi)
↓
Hardware (Donanım: CPU, RAM, Disk)
Yani sunucuyu aldın, üzerine mesela Windows Server kurdun, onun üzerine de SQL Server'ı kurdun. Bitti. Başka bir işletim sistemi (örneğin Linux) çalıştıramazsın — donanımın tamamı tek işletim sistemine ait.

Sorun neydi?
Donanımlar geliştikçe (64 GB RAM, 32 çekirdek CPU gibi) çoğu uygulama bu gücün küçük bir kısmını kullanıyordu:

CPU kullanımı : %10
RAM kullanımı : %15
Disk kullanımı : %20
Geri kalan kaynak boşa gidiyordu. Pahalı donanım alıyorsun ama büyük kısmı atıl duruyor — ciddi bir verimsizlik.

Buradan şu fikir doğdu: "*Tek bilgisayarda neden birden fazla işletim sistemi çalıştırmayalım?*" İşte bu sorunun cevabı **Sanallaştırma (Virtualization)**.

2. Sanallaştırma (Virtualization)

Artık akış bir katman daha kalınlaştı:

Application (Uygulama)
↓

Guest OS (Ubuntu / Windows / CentOS)
↓
Virtual Machine (Sanal Makine)
↓
Hypervisor
↓
Hardware (Donanım)
Hypervisor buradaki kilit kavram: fiziksel donanımı (CPU, RAM, Disk) yöneten ve bunu birden fazla sanal makineye paylaştıran yazılım. Örneğin 64 GB RAM + 32 CPU'luk bir sunucuyu şöyle bölebilir:

Fiziksel Sunucu (64 GB RAM, 32 CPU)
│
├── VM1 → Ubuntu (8 GB RAM, 4 CPU)
├── VM2 → Windows (16 GB RAM, 8 CPU)
└── VM3 → CentOS (32 GB RAM, 16 CPU)
Her VM aslında **bağımsız, tam bir bilgisayar gibi**: kendi kernel'i, kendi işletim sistemi, kendi driver'ları, kendi kütüphaneleri var. Ubuntu VM'i Windows VM'inden tamamen habersiz ve bağımsız çalışır.

Avantajı: "1 sunucu = 1 işletim sistemi" yerine "1 sunucu = N işletim sistemi" oldu. Donanım artık ihtiyaç kadar paylaştırılıyor.

Ama sorunu da var: Her VM için ayrı ayrı işletim sistemi kurulmalı, kernel çalışmalı, güncelleme yapılmalı, lisans ve güvenlik yönetilmeli, disk alanı harcanmalı. Yani her VM gerçekten *tam bir bilgisayar* — bu da fazla RAM, fazla CPU, fazla disk ve fazla yönetim yükü demek.

Ayrıca bir uygulama CPU istediğinde isteğin donanığa ulaşması için geçtiği katman sayısı arttı:

Application → Guest OS → Hypervisor → (bazen Host OS) → Hardware
Katman arttıkça performans kaybı ve yönetim yükü de artıyor.

Eğitmenin asıl vurgusu

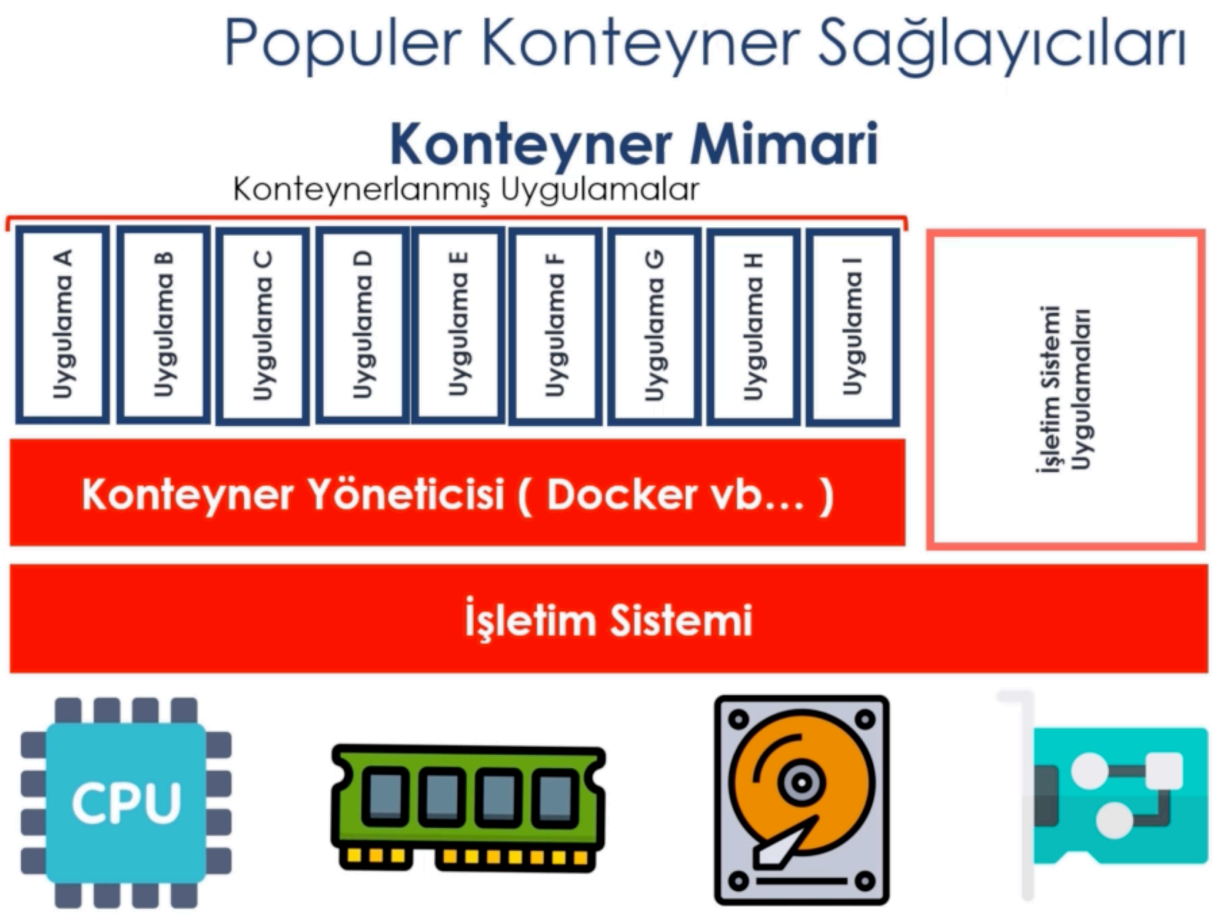
"Bizim amacımız işletim sistemi kurmak değil, uygulamamızı güvenli ve sorunsuz çalıştıracak bir ortam oluşturmaktır."

Yani kullanıcı için önemli olan işletim sistemi değil, **uygulamanın çalışması**. Docker ve container mimarisi tam bu noktada devreye giriyor: her uygulama için ayrı bir işletim sistemi kurmak yerine, sadece uygulamanın ihtiyacı olan ortamı izole ederek çok daha hafif ve hızlı bir çözüm sunuyor.

Kısa özet

Mimari	Özellik	Zayıf yönü
Geleneksel	1 sunucu = 1 OS	Donanım verimsiz kullanılır
Sanallaştırma	Hypervisor ile 1 sunucu = N OS	Her VM ağır (kendi kernel'i, kaynak tüketimi yüksek)

Container (sıradaki konu)	Sadece uygulama ortamı izole edilir	Docker'ın çözdüğü nokta



Container nedir?

Fikir şu: "*Uygulamanın çalışması için gereken her şeyi tek pakette toplayalım*." Bu pakete **Container** deniyor — kod, kütüphaneler, runtime, konfigürasyon, tüm bağımlılıklar bir arada.

Neden kullanılır?

Geliştirici Bilgisayarı → Test Ortamı → Production → Cloud
Container her ortamda aynı şekilde çalışır, çünkü ihtiyacı olan her şey içinde. Bu yüzden "benim bilgisayarımda çalışıyordu" problemi büyük ölçüde ortadan kalkar.

Container nasıl çalışır?

Container tek başına çalışamaz — onu yöneten bir motor gerekir: **Docker Engine**.

Application → Container → Docker Engine → Host OS → Hardware
Docker Engine container'ı oluşturur, çalıştırır, durdurur, siler, yönetir.

VM ile fark (en kritik nokta)

Eskiden her servis ayrı bir VM'e kuruluyordu:

VM1 → Ubuntu → PostgreSQL
VM2 → Ubuntu → Python API
VM3 → Ubuntu → Redis
Her VM kendi OS'unu taşıdığı için ağır, yavaş açılıyor, yönetimi zor.

Docker ise diyor ki: "*Her servis için yeniden Ubuntu kurmana gerek yok*." Tüm container'lar aynı host'un kernel'ini paylaşır:

Host OS
  │
Docker Engine
  │
├── PostgreSQL Container
├── Redis Container
├── API Container
└── Nginx Container
Sonuç: daha hafif, çok hızlı başlıyor, az RAM kullanıyor, aynı sunucuda çok daha fazla container barınabiliyor.

Docker'ın çözdüğü asıl problem

Geliştirici Bilgisayarı → Çalışıyor
Test Ortamı → Çalışıyor
Production → Çalışmıyor

Sebep: ortamlar farklıydı (Python/Java sürümü, Linux sürümü, kütüphaneler...). Docker, geliştirmede oluşturulan container'ı test ve production'a aynen taşıdığı için uygulama her yerde aynı ortamda çalışır.

Üç mimari kıyası

Geleneksel: Application → OS → Hardware
Sanal (VM): Application → Guest OS → VM → Hypervisor → Hardware
Container: Application → Container → Docker Engine → Host OS → Hardware

Container'ın avantajları

Avantaj	Açıklama
Platform bağımsızlığı	Dev/Test/Prod/Cloud'da değişmeden çalışır
Kaynak verimliliği	Ayrı OS gerekmediği için az RAM/CPU/disk
İzolasyon	Bir container çökerse diğerleri etkilenmez (tam güvenli değil ama)
Hız	Saniyeler içinde oluşur/başlar/silinir — çünkü yeni OS açılmaz
Operasyonel	Host güncellenirken container'lar etkilenmez
Ölçeklenebilirlik	Yoğunlukta (örn. Black Friday'de 2→8 container) hızlıca büyütülüp küçültülebilir
CI/CD kolaylığı	Kod→Test→Build→Production aynı container ile ilerler, ortam farkı hatası azalır

Docker mı, VMware mi?

En kritik mesaj: **rakip değiller, farklı problemleri çözüyorlar.**

VM (VMware): donanımı sanallaştırır. Sanki yeni bir fiziksel bilgisayar yaratıp üzerine OS kurarsın; Ubuntu kendisinin sanal olduğunu bile bilmez.

Hardware → Hypervisor → Virtual Machine → Operating System → Application
Docker: donanımı taklit etmez, mevcut OS'un kaynaklarını paylaşarak uygulamaları izole eder.

Hardware → Host Operating System → Docker Engine → Container → Application
Docker daha hızlı çünkü VM'de önce OS kurulup kernel başlıyor (dakikalar sürebilir), Docker'da sadece uygulama başlıyor (saniyeler).

Docker, VMware'in yerini alır mı? Hayır — amaçları farklı: VM farklı işletim sistemleri çalıştırmak için, Docker uygulamaları paketleyip taşınabilir/izole hale getirmek için var.

Birlikte kullanılabilirler mi? Evet, hatta büyük şirketlerde en yaygın senaryo bu:

Fiziksel Sunucu → VMware → Ubuntu VM → Docker Engine → Container'lar (API, Redis, PostgreSQL...)
Hatta VMware bile artık Docker ve Kubernetes'i destekliyor — onu rakip değil, tamamlayıcı görüyor.

Container Engine Nedir?

Container'lar kendi başına çalışamaz, onları yöneten bir yazılım gerekir → **Container Engine**.

Container → Container Engine → Host Operating System
Görevi: container oluşturur, başlatır, durdurur, siler, yönetir.

Popüler Container Engine'ler: Docker, Linux Containers (LXC), Windows Server Containers → en yaygını Docker.

Container teknolojisinin tarihçesi

Önemli düzeltme: **Docker container teknolojisini icat etmedi.**

2000 FreeBSD Jails → 2001 Linux izolasyon çalışmaları → 2004 Solaris Containers
→ 2005 OpenVZ → 2006 Google Process Containers → 2007 cgroups
→ 2008 LXC → 2013 Docker
Docker, 2013'te LXC üzerine geliştirmeler yaparak kendi mimarisini kurdu, kullanımı kolaylaştırdı ve container'ın yaygınlaşmasını sağladı.

Microsoft neden Docker'ı destekledi?

Docker Linux'ta hızla yayılınca Microsoft da bunu Windows'a getirmek istedi:

- **Windows Server 2016** → ilk container desteği geldi ama tam kararlı değildi
- **Windows Server 2019** → daha stabil hale geldi

Linux Container vs Windows Container

Container kendi OS'unu taşımaz, **host'un kernel'ini** kullanır → kernel aynı olmalı:

Windows Host → Windows Container ✔ Windows Host → Linux Container ✘
Linux Host → Linux Container ✔ Linux Host → Windows Container ✘
Peki Windows'ta Linux container nasıl çalışıyor? Docker Desktop arka planda küçük bir Linux VM çalıştırır:

Windows → Docker Desktop → Küçük Linux VM → Docker Engine → Linux Container
Aynı mantık macOS için de geçerli.

Docker komutları aynı mı?

Evet — `docker run`, `docker ps`, `docker images`, `docker stop` Windows'ta da Linux'ta da aynı şekilde çalışır. Tek fark: kullanılan image'ın istediği kernel ailesi (Linux image → Linux kernel, Windows image → Windows kernel).

DevOps ile ilişkisi

DevOps kültürü yaygınlaştıkça şirketler; uygulamaları hızlı dağıtmak, yönetimi kolaylaştırmak ve standart bir geliştirme ortamı kurmak için container teknolojisini yoğun kullanıyor. Bu yüzden Docker, yazılım geliştirme süreçlerinin önemli bir parçası haline geldi.

Docker Nedir?

Docker, container teknolojisini kullanarak uygulama geliştirme/test/dağıtımını kolaylaştıran **açık kaynaklı bir Container Engine platformudur**.

Application + Runtime + Libraries + Config Files + Dependencies
↓
Docker
↓
Container

Docker neden bu kadar popüler oldu?

LXC (Linux Containers) → Docker → Container kullanımı yaygınlaştı
Docker, LXC'nin üzerine kurularak container kullanımını çok daha kolay hale getirdi.

Docker'ın sağladığı kolaylıklar

Uygulamalar kolayca paketlenir, hızlıca test edilir, farklı ortamlara taşınır, aynı anda çoğaltılır, saniyeler içinde çalıştırılır. Örnek — trafik arttığında (kampanya günü gibi) Docker ihtiyaç kadar yeni container oluşturur:

Normal Gün: API, API
İndirim Günü: API, API, API, API, API, API, API, API

Docker açık kaynaklıdır

- İlk sürüm (Docker 1.0) **2014**'te yayınlandı.
- Google, Microsoft, IBM gibi büyük şirketler tarafından yoğun kullanılıyor → bu da Docker'ın standart haline gelmesinin en önemli sebeplerinden biri.

Genel çalışma mantığı

Uygulama → Docker → Container → Docker Engine → Host Operating System → Hardware

Docker ile Container aynı şey mi?

Hayır. Container = uygulamanın paketlenmiş hali. Docker = bu container'ları oluşturan/yöneten platform.

Container = Kargo Kutusu
Docker = O kutuyu hazırlayan, taşıyan, yöneten lojistik şirketi

Docker olmasa da başka Container Engine'lerle container çalıştırılabilir; ama en yaygın araç Docker olduğu için ikisi genelde birlikte anılır.

Docker Mimarisi

Bileşen	Görev
Docker Client	Komutu alır, Daemon'a iletir, sonucu gösterir (garson gibi — sipariş alır, mutfağa iletir)
Docker Daemon	Docker'ın beyni: image indirir, container/network/volume oluşturur, başlatır, durdurur
Docker Host	Docker'ın çalıştığı makine — kendi bilgisayarın, bir VM veya fiziksel sunucu olabilir
Docker Registry	Image deposu (GitHub'ın kod deposu olması gibi) — Ubuntu, MySQL, Redis, PostgreSQL, Nginx, WordPress gibi binlerce hazır image barındırır
Docker Image	Container oluşturmak için hazır şablon (template). Çalışmaz, sadece bekler — Windows ISO dosyası gibi düşün: kurulum için kullanılır ama kendisi çalışmaz
Docker Container	Image'ın çalışan hali

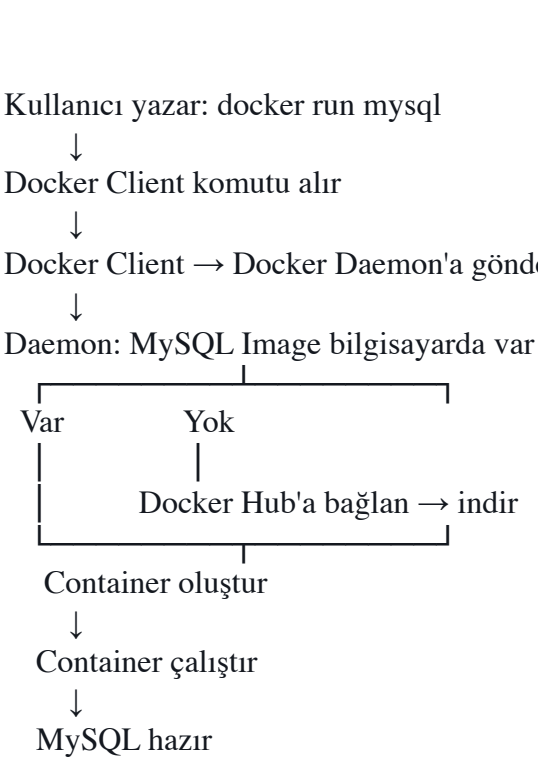
Image ──docker run──▶ Container
Aynı image'dan istediğin kadar container oluşturabilirsin:

MySQL Image → Container 1, Container 2, Container 3...

Docker Engine'in içi

Docker Engine
├── Docker Client → komutu alır
├── REST API → Client ile Daemon'un haberleşmesini sağlar
└── Docker Daemon → işlemleri gerçekleştirir

`docker run mysql` yazınca ne oluyor? (tüm akış)



Linux Kernel — Docker'ın temeli

Docker, Linux Kernel'in iki özelliğini kullanır:

- **Namespaces:** container'ları birbirinden izole eder
- **cgroups:** CPU, RAM ve diğer kaynak kullanımını sınırlar

Tek cümlelik özet: Kullanıcı Docker Client üzerinden komut gönderir → Daemon işler → image yoksa Docker Hub'dan indirir → image'dan container oluşturup çalıştırır → uygulama Docker Engine üzerinde izole çalışır.

Namespaces ve cgroups (detaylı)

Docker, VM'lerdeki gibi her uygulama için ayrı işletim sistemi kurmaz. Bunun yerine Linux Kernel'in iki teknolojisini kullanır.

Namespaces — "neyi görebilir?"

Container'ın sistem üzerinde neyi göreceğini belirler. Bir MySQL container'ı: sadece kendi process'lerini, kendi dosya sistemini, kendi network alanını görür — diğer container'ların varlığından haberdar olmaz.

Benzetme: özel gözlük — aynı odada birçok kişi var ama herkesin gözlüğü sadece kendi masasını gösteriyor.

Namespace türü	İzole ettiği
PID Namespace	Çalışan process'ler
Mount Namespace	Dosya sistemi / klasörler

Network Namespace	Ağ arayüzleri
User Namespace	Kullanıcılar ve yetkiler
UTS Namespace	Hostname bilgisi

PID Namespace örneği: Host'ta yüzlerce process çalışsa da, container içinde sadece o container'a ait process'ler (`mysqld`, `bash` gibi) görünür.

Mount Namespace örneği: Host'un dosya sistemi `/home, /etc, /var, /usr, /opt` gibi klasörlere sahipken, container'a sadece `/app` gösterilebilir — container bunu gerçek kök dizin sanır, üstteki host klasörlerinden habersizdir.

cgroups — "ne kadar kullanabilir?"

Container'ın CPU, RAM, disk, network kullanımını sınırlar. Sınır konmazsa bir container tüm RAM/CPU'yu tüketip diğer uygulamaları yavaşlatabilir.

Örnek sınırlar: max 2 GB RAM, max 2 CPU çekirdeği.

Docker'ın görevi

Kullanıcı → Docker → Linux Kernel → Namespaces (izolasyon) → cgroups (kaynak sınırı)
→ Image'dan Container oluşur → Uygulama çalışır
Docker yeni bir izolasyon teknolojisi icat etmedi — Kernel'de zaten var olan bu özellikleri kolay kullanılabilir hale getirdi.

Ezber: Namespaces = neyi görebilir? / cgroups = ne kadar kullanabilir? / Docker = bu ikisini yöneten araç.

Docker Kurulum Modelleri

	CE (Community)	EE (Enterprise)
Fiy	Ücretsiz, açık kaynak	Ücretli
Ki min	Öğrenci, geliştirici, küçük/orta proje	Büyük şirket, production
İçer diği	Container/image oluşturma, Docker Hub,	CE'nin tamamı + resmî teknik destek, sertifikalı image, merkezi yönetim, kurumsal güvenlik politikaları, Windows/Linux/bulut'u

Hangisini seçmeli? Yeni öğreniyorsan / kendi bilgisayarındaysan / küçük projedeysen → CE yeterli. Kurumsal destek, gelişmiş güvenlik, sertifikalı image zorunluluğu, büyük ölçekli production → EE.

CE'nin alt sürümleri

- **Stable:** test edilmiş, kararlı, ~3 ayda bir yayınlanır, günlük kullanım/production için önerilir.
- **Edge:** beta/deneme, yeni özellikler önce burada çıkar, ~ayda bir yayınlanır, hata içerebilir.

Sürüm numarası formatı

Yıl.Ay.Patch → örnek: 24.0.5
(Not: Günümüzde numaralandırma değişti, bu mantık genel fikir vermek için.)

Docker Demo (iki örnek)

IIS Container: Normalde OS kurulumu + ayarlar + IIS servis kurulumu + dosya yerleştirme gerekirken, tek komutla saniyeler içinde web sunucusu ayağa kalkıyor (localhost önce boş, komut sonrası çalışıyor).

SQL Server Container: Normalde kurulum dosyası indirme + yapılandırma + servis kurulumu gerekirken, tek komutla SQL Server container'ı oluşturuluyor. SSMS ile **1433** portundan bağlanılıp yeni bir database oluşturularak sistem doğrulanıyor.

Amaç: Docker sıfırdan kurulum yapmıyor — hazır **Image**'ları kullanıyor. Image içinde uygulama, runtime, kütüphaneler, konfigürasyon, bağımlılıklar zaten hazır; Docker Engine bunu kullanarak saniyeler içinde çalışan container üretiyor.

Docker'ın Çalıştığını Kontrol Etme

docker version → Client ve Server (Engine/Daemon) bilgisi verir.
Hem Client hem Server görünüyorsa Docker sorunsuz çalışıyor demektir.
Server görünmüyorsa Engine çalışmıyor ya da bağlantı sorunu var.

docker info → toplam/çalışan/duran container sayısı, image sayısı,
Storage Driver, OS/kernel bilgisi, CPU/RAM bilgisi

docker --help → tüm temel komutları listeler
docker run --help → belirli bir komut hakkında bilgi verir
Komut yapısı: Eski `docker run` hâlâ çalışır ama günümüzde nesne tabanlı yeni yapı tercih edilir:

```
docker container run
   |       |     |
 program  nesne  işlem
```

İlk Container Oluşturma (Hello World)

docker container run = docker container create + docker container start
(`run` önce container'ı **oluşturur**, sonra hemen **başlatır**.)

Container mutlaka bir Image'dan oluşturulur:

docker container run hello-world
docker container run alpine
docker container run python
docker container run python:2 ← belirli sürüm
docker container run alpine:3.9
`latest` etiketi: Sürüm belirtilmezse Docker otomatik `latest` kullanır → `python` = `python:latest`.

Docker arka planda ne yapıyor?

docker container run hello-world
↓
Image bilgisayarda var mı?
Evet → direkt o image'dan container oluştur
Hayır → Docker Hub'a git → image indir → container oluştur
↓
Container çalışır
Docker Hub: hello-world, alpine, python, nginx, mysql gibi binlerce hazır image barındıran çevrim içi depo.

Oluşturulan container'a atanan bilgiler

Container ID, Image, Command (başlangıçta çalıştırılan komut), Status, Name (otomatik verilir).

İsim vermek:

docker container run --name proje1 hello-world
Kek tarifi benzetmesi: Image = tarif (uygulamayı çalıştırmak için gereken tüm dosya/bağımlılık/ayar), Docker bu tarifi kullanarak çalışan container'ı (gerçek keki) oluşturur.

Docker Container Yönetimi

Oluşturma

docker container run <image>
Docker önce image'ı yerelde arar → yoksa Docker Hub'dan indirir (pull) → container oluşturur → çalıştırır. **Not:** `run`'dan sonra geçerli bir image adı yazılmalı; rastgele isim yazılırsa Docker Hub'da arar, bulamazsa hata verir.

Listeleme

docker container ls → çalışan container'lar
docker container ls -a → tüm container'lar (durmuş dahil)
docker container ls -q → sadece ID'ler
Gösterilen bilgiler: Container ID, Image, Command, Status, Port, Name.

Container'ın çalışma mantığı

Command'da belirtilen başlangıç komutu tamamlanınca container **Exited** durumuna geçer.

- Kısa işlem yapıp duranlar: `hello-world`, `python`, `alpine`
- Sürekli çalışanlar (Up durumunda kalır): `nginx`, `mysql`, `docs/docker.github.io`

Yönetim komutları

docker container start <c> → başlat
docker container stop <c> → durdur
docker container restart <c> → yeniden başlat
docker container pause <c> → duraklat
docker container unpause <c> → devam ettir
docker container rename eski yeni → isim değiştir

Silme

docker container rm <c> → sil (önce durdurulmalı)
docker container rm -f <c> → zorla sil (durdur + sil, -f = force)
docker container rm id1 id2 id3 → toplu sil
docker container rm $(docker container ls -aq) → tüm container'ları sil
docker container prune → sadece durmuş olanları temizler (çalışanlara dokunmaz)

